

Edge Impulse Response

1. Dataset Context and Composition

1.1 Dataset Overview

The training dataset consists of 92,800 labeled samples collected from accelerometer and gyroscope sensors mounted on dog collars. Data was collected at 20Hz sampling rate in 3.2-second windows (64 samples per window).

Activity Class	Original Samples	After Augmentation	Percentage
standing	24,347	15,000	14.5%
walking	22,891	15,000	14.5%
lying	13,409	13,409	13.0%
sitting	11,842	12,000	11.6%
running_playing	9,128	12,000	11.6%
eating_drinking	8,547	12,000	11.6%
human_carry	1,789	12,000	11.6%
on_shelf	847	12,000	11.6%
TOTAL	92,800	103,409	100%

1.2 Critical Dataset Limitations

Controlled Environment Bias: All data collected in controlled settings with researchers present. Dogs aware of monitoring, leading to more "compliant" behaviors than natural daily activity.

Collar Orientation Assumption: Training data assumes collar is properly positioned with sensor on top of neck. No samples collected with rotated or flipped collars. This is THE critical limitation.

Limited Dog Diversity: Data from ~10-15 individual dogs, primarily medium-sized breeds. Limited coverage of very small (<5kg) and very large (>40kg) dogs.

Class Imbalance: Original data showed 28.8:1 ratio (standing: 24,347 vs on_shelf: 847). Despite augmentation reducing this to 1.25:1, minority classes still suffer from limited real behavioral diversity.

Edge Case Underrepresentation: Rare behaviors like scratching while standing, play bows, stretching, head shaking, and transitional movements are underrepresented or absent.

1.3 Data Augmentation Strategy: Two-Level Approach

The augmentation approach operates at **two distinct levels** - raw time-series augmentation and feature-level augmentation - forming a comprehensive two-stage pipeline designed to address severe class imbalance while maintaining physical plausibility. At the **raw sensor data level**, a suite of sophisticated time-series augmentation techniques is available including jittering (adding Gaussian noise to sensor readings with controlled standard deviation), scaling (multiplying sensor values by random factors to simulate sensor calibration variations or different movement intensities), rotation (randomly permuting and flipping sensor axes to simulate collar orientation changes), time warping (non-linear temporal stretching and compression using cubic spline interpolation to simulate variations in movement speed), magnitude warping (applying smooth amplitude modulations via cubic splines to capture intensity variations within movements), window slicing (extracting and interpolating temporal subsequences to create valid-length samples from variable-duration activities), window warping (locally stretching or compressing specific temporal regions to simulate irregular pacing), permutation (reordering temporal segments to break strict temporal dependencies while preserving local patterns), and advanced techniques like SPAWNER and discriminative guided warping that use Dynamic Time Warping (DTW) to find optimal alignments between samples of the same class and generate synthetic trajectories. These raw-level augmentations operate on the six-channel time series (three accelerometer axes, three gyroscope axes) before any feature extraction occurs, and are particularly powerful because they generate entirely new temporal patterns that, when passed through the feature extraction pipeline, produce feature vectors that may explore regions of feature space not directly covered by simple feature-space noise addition. However, raw-level augmentation must be applied carefully because naive manipulations can easily violate physical constraints - for instance, scaling can change gravity magnitude away from 1g, rotation can misalign gravity with actual orientation, and excessive jittering can introduce frequencies above the Nyquist limit - which is why the parameters (sigma values, scaling factors, warping knots) are carefully tuned to maintain physical plausibility, and why these techniques are typically applied selectively rather than to all samples.

Following raw-level augmentation (or in parallel for certain classes), **feature-level augmentation** provides a complementary approach using the **class-aware, three-tier strategy** tailored to each activity's data availability and physical characteristics. For **large classes** (standing, walking, lying with tens of thousands of samples), we apply **downsampling only** to prevent model bias, avoiding both raw and feature-level synthetic generation to preserve authentic motion patterns. For **medium classes** (sitting, running_playing, eating_drinking with moderate representation), we perform **minimal feature-space noise augmentation** after feature extraction, adding Gaussian noise proportional to a small percentage of each feature's observed standard deviation computed from real samples; this conservative approach introduces slight variations without distorting the underlying statistical structure, and complements any raw-level augmentation that may have been applied earlier in the pipeline. For **critical minority classes** (on_shelf and human_carry with severely limited samples), we employ **physics-based, feature-targeted augmentation** that operates directly in feature space by sampling from statistical distributions derived from real data analysis - for example, on_shelf augmentation targets spectral entropy features to maintain characteristically low values indicating stable resting posture, mean crossing rate features to preserve the subtle periodicity of breathing and micro-adjustments, and spectral centroid features to reflect the concentration of minimal movement energy; human_carry augmentation injects periodic sinusoidal components with random phase offsets into gyroscope-related features to simulate the characteristic rhythm of human bipedal gait while the dog remains passive. This feature-level augmentation has the critical advantage of **automatically preserving physical constraints** because features like correlation coefficients are inherently bounded, spectral properties satisfy Parseval's theorem, and gravity-derived features maintain unit magnitude, whereas raw-level augmentation requires explicit constraint checking.

The **combination of both levels** creates a powerful synergy: raw-level augmentation generates diverse temporal patterns and movement variations that expand coverage of the time-series manifold, while feature-level augmentation fills gaps in the feature-space representation and enables targeted generation for underrepresented classes where collecting more real data is impractical. In practice, the pipeline applies raw-level augmentation selectively (often to medium classes or specific scenarios like simulating collar rotation), extracts features from both original and augmented time series, then applies the three-tier feature-level strategy to balance the final dataset, reducing class imbalance from an extreme ratio exceeding 25:1 to a manageable ratio of approximately 1.25:1. The decision of which augmentation level to use depends on the class and available data: raw-level techniques like time warping and DTW-based methods are computationally expensive and most beneficial when applied to medium-sized classes where diverse temporal patterns exist but more variability is needed, while feature-level augmentation is computationally cheaper and can generate large numbers of synthetic samples for critical minorities where even raw augmentation would struggle due to insufficient seed samples. Importantly, both approaches are applied with physical awareness - raw augmentation parameters are tuned to avoid violating sensor physics (gravity magnitude, frequency limits, physically impossible accelerations), while feature-level augmentation inherits physical validity from the feature computation process itself, ensuring that every synthetic sample, regardless of generation method, represents a configuration that could theoretically arise from real sensor measurements of plausible dog movements, thus maintaining the model's ability to generalize to real-world deployment scenarios.

2. Orientation Computation Methodology

2.1 Sensor Coordinate System

The 6-axis IMU provides:

- 3-axis accelerometer: Measures both dynamic acceleration (movement) and static gravity (1g toward Earth)
- 3-axis gyroscope: Measures angular velocity (rotation rates)

Assumed sensor orientation (CRITICAL ASSUMPTION):

- X-axis: Lateral (left-right across dog neck)
- Y-axis: Longitudinal (head-tail direction)
- Z-axis: Dorsoventral (back-belly, pointing up from collar)

2.2 Gravity Separation (Butterworth Filter)

Static gravity is separated from dynamic body acceleration using a 4th-order Butterworth low-pass filter:

- Cutoff frequency: 0.3 Hz
 - Rationale: Dog movements typically occur at 0.5-10 Hz; gravity is DC (0 Hz)
- Filter order: 4th-order (steeper roll-off, better separation)
- Sampling rate: 20 Hz

Mathematical formulation:

$\text{gravity_x} = \text{LPF}(\text{accel_x}, f_{\text{cutoff}}=0.3\text{Hz})$

$\text{body_x} = \text{accel_x} - \text{gravity_x}$

This separates:

- Gravity components (static): Encode body orientation relative to Earth
- Body components (dynamic): Encode movement patterns

2.3 Orientation Features

Key gravity-based features used for orientation detection:

x_gravity_mean: Lateral gravity component. Standing/Sitting 0.0g (upright), Lying +/- 0.95g (on side)

y_gravity_mean: Longitudinal gravity. Usually 0g for all postures (spine horizontal)

z_gravity_mean: Vertical gravity. Standing 1.0g (back faces up), Lying 0.1g (back sideways)

xz_gravity_correlation: Captures body angle. Sitting shows positive correlation (cca 0.3) due to forward tilt

2.4 THE CRITICAL FLAW: Orientation Assumption Breaks in Real World

The entire feature engineering approach assumes the collar maintains a fixed orientation. In reality:

- Collar rotates during normal wear: Dogs scratch, shake, roll, and swim
 - Sensor can flip 180° (underneath neck)
 - Sensor can rotate 90° (on side of neck)
- Collar looseness varies: Tight (1 finger) vs loose (3+ fingers)
- Owners may not position collar consistently

Impact on gravity features when collar rotates 90°:

- x_gravity_mean (lateral) becomes z_gravity_mean (vertical)
 - Standing dog now reads x_gravity 1.0g instead of 0.0g
 - Model thinks dog is LYING when actually STANDING

THIS IS NOT THE REASON FOR THE SIGNIFICANT PERFORMANCE DROP IN QA TEST.

3. Model Training Workflow

3.1 End-to-End Training Pipeline

Step 1: Data Collection and Preprocessing

- Raw sensor data collected at 20Hz (accel_x, accel_y, accel_z, gyro_x, gyro_y, gyro_z)
- Segmented into 3.2-second windows (64 samples per window)
- Manual labeling by researchers watching video recordings
- Quality control: Windows with ambiguous behavior discarded

Step 2: Feature Extraction

- Gravity separation using Butterworth filter
- 420+ candidate features computed per window:
 - Time-domain: mean, std, skewness, kurtosis, RMS, crest factor
 - Frequency-domain: FFT, spectral entropy, spectral spread, dominant frequency
 - Correlations: Inter-axis correlations (xy, xz, yz for accel and gyro)
 - Jerk: Rate of change of acceleration/rotation

Step 3: Feature Selection

- Train Random Forest on all 420+ features
- Compute feature importance using mean decrease in impurity
- Select top 50 features (optimal accuracy vs. RAM constraint trade-off)
- Export feature list to feature_importance_best_fold.csv

Step 4: Data Augmentation (Three-Tier Strategy)

- Tier 1 (Large classes): Downsample to 15,000 samples (standing, walking, lying)
- Tier 2 (Medium classes): Minimal noise augmentation (sitting, running, eating)
- Tier 3 (Minorities): Physics-based augmentation (on_shelf, human_carry)
 - Result: 28.8:1 imbalance 1.25:1 balanced dataset

Step 5: Model Training

- Architecture: ResNet-MLP
- Loss function: FocalLoss
- Optimizer: Muon
- Training: 5-fold cross-validation, 500 epochs per fold
- Early stopping: Patience=50 epochs based on validation accuracy
- BatchNorm calibration

Step 6: Model Validation

- Test on held-out fold (20% of data per fold)
- Metrics: Overall accuracy (97.6%), per-class precision/recall/F1
- Confusion matrix analysis to identify systematic errors

3.2 Training Results Summary

Metric	Value
Overall Accuracy (PyTorch)	97.6%
Overall Accuracy (STM32)	97.54%
PyTorch STM32 Gap	0.06%
Lowest Class Accuracy	eating_drinking: 91.2%
Highest Class Accuracy	lying: 98.1%
Real-World Accuracy (QA)	cca 60% (from Michael)

4. Python to C Conversion Process

4.1 Conversion Workflow

The conversion from Python to C happens in two parallel streams:

Stream 1: Model Weights (PyTorch ONNX STM32Cube.AI C)

1. Export trained PyTorch model to ONNX format
 - Tool: torch.onnx.export() with opset_version=11
 - BatchNorm layers fused into linear layers
1. Import ONNX into STM32Cube.AI tool
 - Quantization: "low compression"
 - Reason: int8 quantization causes 10% accuracy drop (95% 85%)
1. STM32Cube.AI generates C code for inference
 - Output: activity_recognition.c, activity_recognition.h, weights array

Stream 2: Feature Computation (Python Manual C Port)

1. Python feature functions translated manually to C
 - Libraries used: ARM CMSIS-DSP for FFT, math.h for standard functions
1. Validation: Same raw sensor window processed in Python and C
 - Compare 50 feature outputs element-by-element
 - Acceptance: Absolute difference <0.01 for all features

4.2 Validation Results

- Feature computation: All 50 features match Python (max difference: 0.0051)
- Model inference: PyTorch (97.6%) vs STM32 (97.54%) = 0.06% gap

Conclusion: PythonC conversion is correct. The 60% real-world performance is NOT a conversion issue.

5. Integration and Inference Pipeline

5.1 System Architecture

The embedded inference pipeline operates in real-time at 20Hz:

- 1. Sensor Acquisition
- 2. Feature Computation
- 3. Feature Normalization
- 4. Model Inference
 - Feed 50 normalized features into STM32Cube.AI generated network
 - Forward pass: Input BN 3×ResBlock Output
 - Softmax outputs: 8 class probabilities
- 5. Post-Processing
 - Argmax: Select class with highest probability
 - Optional: Temporal smoothing (majority vote over 3 classifications)
 - Output: Class label + confidence

6. Performance Gap Analysis: 97% 60%

CRITICAL FINDING: The pipeline is working correctly. Lab performance (97.54%) matches Python (97.6%). The issue is a fundamental train-test distribution mismatch.

6.1 What We Know Works

- Feature computation: Python vs C difference <0.001
- Model conversion: ONNX export validated, weights match
- Inference pipeline: STM32 achieves 97.54% on test set
- Hardware performance: Latency, memory, power all within spec

6.2 Observed Real-World Behavior

QA team reports from field testing:

- Standing frequently misclassified as lying (collar rotated)
- Walking confused with running during excited movement
- Eating misclassified as standing when dog pauses between bites
- Scratching while standing classified as running/playing
- Head shaking causes brief "explosion" of random classifications
- Performance degrades significantly with loose-fitting collars

6.3 Estimated Contribution to cca 35% Accuracy Loss

Factor	Est. Impact	Confidence
Collar orientation variability	15-20%	HIGH
Edge case behaviors not in training	8-12%	HIGH
Behavioral diversity (natural vs controlled)	5-8%	MEDIUM
Collar fit variability	3-5%	MEDIUM
Environmental factors (temperature, terrain)	2-4%	LOW
Class imbalance effects	2-3%	LOW
TOTAL ESTIMATED LOSS	35-52%	(overlaps exist)

7. Root Cause Analysis

Primary Root Cause: Orientation-Dependent Features with No Orientation Normalization

The model relies heavily on gravity-based features that assume fixed sensor orientation:

- x_gravity_mean, y_gravity_mean, z_gravity_mean (3 of top 10 features)

- xz_gravity_correlation, xy_gravity_correlation (2 of top 50 features)
- These features lose ALL discriminative power when collar rotates

Example scenario:

1. Dog is standing (training assumption: sensor on top of neck)
 - Features: x_gravity 0.0g, z_gravity 1.0g Model predicts "standing"
1. Collar rotates 90° during play (sensor now on side of neck)
 - Features: x_gravity 1.0g, z_gravity 0.0g Model predicts "lying"
1. Model is systematically wrong for cca 20-30% of real-world samples

Why this was not caught in testing:

- Test set came from same controlled data collection same orientation assumption
- Lab testing: Researchers carefully position collars assumption holds
- Real world: Dogs rotate collars constantly assumption violated